

Estado del Arte: IA Generativa en Ingenieria de Software 2025

TechPulse Research - Edicion Q2 2025

Analisis basado en 4,200 encuestados en LATAM y Espana

Este reporte examina como los equipos de desarrollo en Latinoamerica y Espana adoptan herramientas de IA generativa. Cubre adopcion de LLMs, impacto en productividad, seguridad y tendencias emergentes como RAG y agentes autonomos.

1. Adopcion de IA Generativa en Equipos de Desarrollo

El 78% de los desarrolladores encuestados reporta usar al menos una herramienta de IA en su flujo diario, frente al 31% en 2023. El crecimiento es especialmente pronunciado en equipos de backend y full-stack.

78%

usan IA a diario
vs 31% en 2023

+42%

mejora productividad
autoreportado

34%

politica IA formal
vs 9% en 2023

Herramientas mas utilizadas

- GitHub Copilot / Copilot Chat - 61%
- ChatGPT / GPT-4 API - 58%
- Claude (Anthropic) - 29%
- Ollama (modelos locales) - 22% (+180% interanual)
- Cursor / Windsurf IDEs IA - 18%

Casos de uso principales

- Autocompletado y generacion de codigo boilerplate (87%)
- Revision de codigo y deteccion de bugs (74%)
- Generacion y mantenimiento de tests unitarios (68%)
- Documentacion automatica de funciones y APIs (61%)
- Consultas sobre documentacion interna via RAG (43%) - mayor crecimiento YoY
- Analisis de logs y diagnostico de incidentes (39%)

2. RAG: el Patron Arquitectonico del Anio

Retrieval-Augmented Generation (RAG) emergio en 2024 como el patron dominante para conectar LLMs a datos privados sin reentrenamiento. En 2025, el 43% de equipos con 20+ ingenieros tiene al menos un sistema RAG en produccion o piloto.

Por que RAG en lugar de Fine-tuning?

- Costo: fine-tuning en GPT-4 puede costar USD 50K a 2M. RAG usa el modelo base sin cambios.
- Velocidad: RAG incorpora nuevos documentos en minutos; fine-tuning requiere horas o dias.
- Trazabilidad: RAG puede citar exactamente el fragmento fuente de cada respuesta.
- Control: los documentos indexados se auditan, actualizan o eliminan facilmente.
- Privacidad: con Ollama + nomic-embed, los datos nunca salen de la infraestructura.

Stack tecnologico mas comun para RAG (2025)

- Vector stores: pgvector/PostgreSQL 38% | Qdrant 22% | ChromaDB 19% | Pinecone 15%
- Embeddings: text-embedding-3-small 41% | nomic-embed-text 28% | mxbai-embed 18%
- LLMs: GPT-4o-mini 44% | Claude 3 Haiku 21% | Llama 3 local 18% | Mistral 12%
- Frameworks: LangChain 52% | LlamaIndex 31% | implementacion propia 17%

Metricas de impacto reportadas

- Reduccion tiempo de onboarding de nuevos devs: -55% en promedio.
- Tickets de soporte interno resueltos sin escalado: +68%.
- Precision sobre documentacion tecnica: 89% de respuestas valoradas correctas.
- Ahorro en busqueda de informacion: 4.2 horas/semana por desarrollador.

3. Modelos Locales con Ollama

Ollama se consolidó como la herramienta de facto para LLMs en infraestructura propia. Un solo binario, soporte para Llama 3, Mistral, Phi-3 y nomic-embed-text. Adopción crítica en fintech, salud y gobierno por privacidad de datos.

Modelos mas utilizados via Ollama

- llama3.2:3b - Q&A livianas, bajo consumo de RAM
- llama3.1:8b - balance optimo calidad/velocidad en hardware consumer
- mistral:7b - favorito para generacion de codigo y razonamiento
- nomic-embed-text - embedding 768 dims, supera text-ada-002 en MTEB
- mxbai-embed-large - 1024 dims, mejor calidad en espaniol y texto tecnico
- phi3:mini - corre en CPU sin GPU dedicada

Benchmarks embedding para documentos en espaniol

Modelo	Dims	MTEB ES	Latencia
nomic-embed-text:v1.5	768	91.2%	12ms
mxbai-embed-large	1024	93.1%	18ms
text-embedding-3-small	1536	94.8%	45ms*
all-minilm (local)	384	85.3%	4ms

** incluye latencia de red a API externa*

Ventajas del stack 100% local

- Cero costo por token: ideal para alto volumen de consultas.
- Latencia predecible: sin dependencia de APIs ni throttling externo.
- Cumplimiento normativo: datos sensibles nunca salen de la red corporativa.
- Stack completo offline: Ollama + pgvector + notebook = RAG 100% local.

4. Seguridad, Desafíos y Tendencias 2025-2026

Principales preocupaciones de seguridad

- Alucinaciones: LLM mezcla contexto recuperado con información inventada (82%)
- Prompt injection en sistemas RAG con inputs no confiables (71%)
- Filtración de información confidencial a través de respuestas (64%)
- Dependencia de APIs externas y riesgo de discontinuación (55%)

Estrategias de mitigación

- Guardrails de output antes de mostrar respuestas al usuario (58%)
- Re-ranking de chunks con cross-encoder (41%)
- Evaluación con RAGAS: Faithfulness, Relevancy, Context Recall (29%)
- Stack local con Ollama para datos sensibles (22%)

Tendencias emergentes 2025

- Agentic RAG: sistemas que planifican múltiples búsquedas autónomamente.
- GraphRAG: enriquecimiento del índice con grafos de conocimiento.
- Hybrid search: combinar vectores (semántica) + BM25 (keywords) para mayor recall.
- Self-RAG: el modelo decide cuando necesita recuperar contexto y cuando no.

Conclusiones

RAG pasó de técnica experimental a estándar de facto para Q&A sobre docs privados. La madurez del ecosistema (pgvector, Ollama, LangChain) reduce la barrera de entrada a días. Los equipos reportan ROI positivo en menos de 3 meses. El stack local Ollama + pgvector es la opción preferida cuando la privacidad es no negociable.